

3 26/01/2021

3.1 Traccia da 12 punti

3.1.1 2pt

Sia data una matrice di interi M (di dimensione $r \times c$). Si scriva una funzione `change` che per ogni cella (i,j) , dove i e j rappresentano rispettivamente l'indice di riga e di colonna, effettui la seguente operazione:

- Se il valore iniziale della cella (i,j) è divisibile per 3, tutta la riga i e la colonna j sono poste al valore 3

```
void change(int **M, int r, int c);
```

Esempio

$$M = \begin{pmatrix} 1 & 1 & 3 & 0 \\ 2 & 4 & 6 & 8 \\ 1 & 2 & 1 & 1 \end{pmatrix} \rightarrow M = \begin{pmatrix} 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 \\ 1 & 2 & 3 & 3 \end{pmatrix}$$

3.1.2 4pt

Definire una struttura dati adeguata a rappresentare una lista singola linkata di interi come ADT I classe, dove il tipo lista si chiami `LIST`. Indicare esplicitamente in quale modulo/file appare la definizione dei tipi proposti. **Non è ammesso l'uso di funzioni di libreria.**

Implementare una funzione caratterizzata dal seguente prototipo:

```
void f(LIST l, int *v, int n)
```

La funzione riceve una lista di interi e un vettore v di interi di n elementi. La funzione cancella dalla lista tutti i nodi contenenti un valore che non appare nel vettore v . **Non è ammesso l'uso di funzioni di libreria.**

Esempio

$L : 3 \rightarrow 10 \rightarrow 6 \rightarrow 3 \rightarrow 1 \rightarrow 4$
 $v = \{1, 2, 3\}$

$L' : 3 \rightarrow 3 \rightarrow 1$

3.1.3 6pt

Data una funzione wrapper con il seguente prototipo

```
void solve(char *str, char **dict, int n, int s);
```

Dove `str` è una generica stringa e `dict` è un vettore di `n` stringhe distinte. Implementare la funzione wrapper `solve` e scrivere una funzione ricorsiva invocata da `solve` che determini se è possibile costruire `str` a partire dalle stringhe del dizionario, anche ripetute, usando almeno `s` stringhe diverse del dizionario. Giustificare la scelta del modello combinatorio adottato. Giustificare la scelta del/dei criteri di pruning adottati, o il motivo della loro assenza.

Esempio

```
str = "abracadabra"  
n = 8  
dict = {"a", "ab", "cada", "abra", "ra", "da", "ca", "bra"}
```

Risultati

```
s = 3 -> a bra cada bra  
s = 4 -> a bra ca da bra
```

3.2 Traccia da 18 punti

3.2.1 Breve introduzione e definizioni

Si vuol realizzare un programma C in grado di gestire il gioco “labirinto”. Un labirinto è composto da stanze collegate direttamente le une alle altre da cunicoli. I cunicoli sono bidirezionali e una data coppia di stanze può essere connessa al più da un cunicolo. Le stanze contengono ricchezze, mentre i cunicoli possono contenere trappole. La mossa elementare è attraversare un cunicolo. Se il cunicolo è dotato di trappola, questa ferisce chi vi passa. Il labirinto può quindi esser visto come un grafo non orientato, i cui i vertici rappresentano le stanze e gli archi i cunicoli. Vertici e archi sono pesati da ricchezze e da assenza/presenza di trappole rispettivamente.

Obiettivo

Un esploratore parte ad esplorare il labirinto avendo a disposizione una sua dote di punti ferita `PF` e un numero di mosse `M` e effettua un percorso nel labirinto con l'obiettivo di accumulare quante più ricchezze possibile, subendo al massimo un numero

di ferite compatibile con PF. Ogni trappola decrementa di 1 i PF dell'esploratore che passa per quel cunicolo. Ogni attraversamento di cunicolo vale una mossa.

Percorso e vincoli

- Entrata/uscita: le stanze sono caratterizzate da una profondità (un intero non negativo assegnato a priori). L'unica stanza a profondità 0 funge da ingresso e uscita dal labirinto; il percorso inizia e termina in questa stanza.
- Percorso: il percorso consiste nel passare da una stanza a un'altra utilizzando i cunicoli. Sono ammessi cicli, cioè si può passare più volte in una stanza (eccetto quella a livello 0, da cui si può solo entrare e uscire) o percorrere più volte un cunicolo.
- Interruzione: un percorso può essere interrotto prima dell'uscita, per uno tra due possibili motivi:
 - raggiungimento del massimo numero di mosse M (ogni attraversamento di cunicolo vale una mossa)
 - esaurimento dei punti ferita (PF): ogni passaggio in un cunicolo con trappola consuma 1 punto ferita (che si sottrae al valore iniziale PF); ci si ferma nella stanza raggiunta se il valore PF dell'esploratore si è azzerato nell'ultimo transito di cunicolo

Ricchezze accumulate

Ogni stanza, eccetto quella di ingresso/uscita, può contenere

- un tesoro, caratterizzato da un valore intero V
- una quantità di oro sparsa per la stanza, descritta dal numero di monete O .

L'esploratore può raccogliere un solo tesoro, mentre non ci sono limiti per l'oro: non è comunque possibile raccogliere più volte l'oro di una stanza, se vi si passa più volte. La ricchezza totale accumulata è la somma dei valori del tesoro e di tutto l'oro eventualmente raccolti (una moneta d'oro ha valore 1).

Conclusione del percorso

- Percorso normale: si raggiunge l'uscita rispettando i vincoli, l'esploratore conquista tutta la ricchezza raccolta.

- Percorso interrotto: se il percorso si è interrotto per raggiunto limite sul numero di mosse o di ferite possibili, ci si ferma nella stanza raggiunta e sono possibili due conclusioni:
 - Soccorso: se la stanza è al livello 1 oppure al 2, arrivano i soccorritori che salvano l'esploratore. I soccorritori ottengono in cambio una parte delle ricchezze accumulate: un terzo se la stanza è al livello 1, metà se al livello 2 (valori da calcolare con arrotondamento per eccesso). Questa parte si sottrae quindi alle ricchezze accumulate;
 - Fallimento: se la stanza è al livello 3 o maggiore, nessuno va a salvare l'esploratore e l'impresa fallisce.

Formato del file

Il labirinto è descritto nel file testuale "labirinto.txt", con il formato seguente:

- la prima riga contiene il numero *S* di stanze
- le successive *S* righe descrivono ognuna una stanza, con formato *<nome> <profondità> <valore_tesoro> <valore_oro>* (Il nome della stanza è una stringa priva di spazi, gli altri campi sono interi non negativi)
- le successive righe (fino alla fine del file) descrivono ognuna un cunicolo, con formato *<nome_stanza_a> <nome_stanza_b> <flag_trappola>* (coppia di nomi di stanze connesse e un flag di valore 0 o 1 per indicare assenza o presenza di trappola)

Segue un esempio di file. I commenti riportati nell'esempio seguente non sono effettivamente contenuti nel file, ma servono solo a rendere più chiara la sua lettura.

```
10
// 10 Stanze totali
Ingresso 0 0 0
// Ingresso sempre a profondità 0 e privo di tesoro e oro
Stanza1 1 0 2
// Stanza1 a profondità 1 senza tesoro e 2 monete d'oro
Stanza2 3 10 3
// Stanza2 a profondità 3 con tesoro di valore 10 e 3 monete
...
// descrizione delle altre 7 stanze
Stanza1 Stanza2 0
// Un cunicolo senza trappola tra Stanza1 e Stanza2
Stanza1 Stanza3 0
// Un cunicolo senza trappola tra Stanza1 e Stanza3
Stanza1 Stanza4 1
```

```
// Un cunicolo con trappola tra Stanza1 e Stanza4  
...
```

Impostazione della soluzione

Si supponga di utilizzare, per affrontare il problema, l'ADT di I classe **Graph** visto a lezione opportunamente modificato. Si realizzino le modifiche e aggiunte richieste. Il **main** non è richiesto, ma andrà aggiunto nella versione allegata alla relazione. Si assuma che i parametri **M** e **PF** siano passati direttamente come argomento al **main**.

3.2.2 Strutture dati

Definire le strutture dati necessarie per rappresentare il labirinto, cioè l'ADT di I classe **Graph**, con le eventuali aggiunte/modifiche proposte per il labirinto con stanze, cunicoli, ricchezze e trappole, suddividendo opportunamente le definizioni tra file header e sorgente. Definire anche una struttura dati **Path**, adatta a rappresentare un percorso effettuato dall'esploratore nel labirinto, comprensivo della sequenza di stanze e delle ricchezze accumulate. Acquisire il labirinto da file in una opportuna struttura dati. Scrivere la funzione:

```
Graph GRAPHload(FILE *fp);
```

La funzione di acquisizione è richiesta in quanto le parti non standard del grafo sono significative.

3.2.3 Problema di verifica

Acquisire da un secondo file un possibile percorso nel labirinto, e verificarne la congruenza. Si chiedono due funzioni:

```
Path GRAPHpathLoad(Graph g, FILE *fp);  
int GRAPHpathCheck(Graph g, PATH p, int M, int PF);
```

La prima funzione acquisisce un percorso da file, che contiene unicamente la sequenza di stanze, identificate ognuna dal nome (quindi mancano ancora le ricchezze accumulate). Il parametro **g** può essere necessario o meno, a seconda di come si sia scelto di definire il tipo **Path**. La seconda funzione verifica un percorso, determinando se rispetti i vincoli: se no, la funzione ritorna 0, se sì, ritorna 1 e calcola e salva per il percorso (parametro **p**) le ricchezze complessive massime accumulabili dall'esploratore.

3.2.4 Problema di ricerca

Individuare una sequenza di mosse all'interno del labirinto (quindi un percorso) tale da massimizzare le ricchezze raccolte compatibilmente con il massimo numero di mosse possibile M e di punti ferita iniziali PF . Si scriva cioè la funzione wrapper e la corrispondente funzione ricorsiva:

```
Path GRAPHpathBest(Graph g, int M, int PF);  
Path GRAPHpathBestR(Graph g, int M, int PF, ...);
```
